

Python for AI & Machine Learning

Class 15 – Polymorphism in Python

Trainer: Surya Dekshith Mallikarjuna

Course: Python for AI & Machine Learning

Duration: 2 Hours (Theory + Practical)

Learning Objectives

After completing this session, students will be able to:

- Understand the concept of Polymorphism.
 - Explain why Polymorphism is one of the four pillars of Object-Oriented Programming.
 - Understand Method Overriding.
 - Understand why Python does not support traditional Method Overloading.
 - Implement Method Overloading using default arguments and `*args`.
 - Understand Duck Typing.
 - Learn Operator Overloading.
 - Apply Polymorphism in AI & Machine Learning and real-world applications.
-

Revision of Previous Class

In the previous class, we learned:

- Inheritance
- Parent and Child Classes
- Constructor Inheritance
- The `super()` function
- Types of Inheritance
- AI examples using Inheritance

Today, we will learn the **third pillar of Object-Oriented Programming (OOP): Polymorphism.**

What is Polymorphism?

The word **Polymorphism** comes from two Greek words:

- **Poly** = Many
- **Morph** = Forms

Polymorphism means "**One Interface, Many Forms.**"

It allows the same method name, function, or operator to perform different actions depending on the object or data it is working with.

Why Do We Need Polymorphism?

Without Polymorphism:

- Different methods are required for similar operations.
- Code becomes repetitive.
- Maintenance becomes difficult.

With Polymorphism:

- Code becomes reusable.
 - Programs become flexible.
 - Maintenance becomes easier.
 - Applications become scalable.
-

Real-Life Example

Consider a person.

The same person can have different roles:

- At home → Parent
- At office → Employee
- At college → Teacher

Although the person is the same, the behavior changes according to the situation.

This is an example of Polymorphism.

Types of Polymorphism in Python

Python supports the following forms of polymorphism:

1. Method Overriding
 2. Method Overloading (using Python techniques)
 3. Duck Typing
 4. Operator Overloading
-

Method Overriding

Method Overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Example

```
class Animal:

    def sound(self):
        print("Animals make sounds")

class Dog(Animal):

    def sound(self):
        print("Dog barks")

class Cat(Animal):

    def sound(self):
        print("Cat meows")

dog = Dog()
cat = Cat()

dog.sound()
cat.sound()
```

Output

```
Dog barks
Cat meows
```

The child classes replace the parent's implementation with their own.

Why Use Method Overriding?

Method Overriding allows child classes to customize inherited behavior without modifying the parent class.

Examples:

- Different AI models implement different training methods.
- Different payment methods process payments differently.
- Different notification systems send notifications differently.

Method Overloading

Method Overloading means creating multiple methods with the same name but different parameter lists.

Languages like Java and C++ support traditional Method Overloading.

Example (Java concept):

```
add(int a, int b)

add(int a, int b, int c)

add(double a, double b)
```

Why Doesn't Python Support Traditional Method Overloading?

Python is a **dynamically typed language**.

Python identifies methods only by their **name**, not by:

- Number of parameters
- Data types of parameters
- Order of parameters

If multiple methods have the same name, the latest method replaces the previous one.

Example

```
class Calculator:

    def add(self, a, b):
        print(a + b)

    def add(self, a, b, c):
        print(a + b + c)

calc = Calculator()

calc.add(10, 20)
```

Output

```
TypeError:  
add() missing 1 required positional argument: 'c'
```

The second `add()` method overwrites the first one.

Achieving Method Overloading Using Default Arguments

```
class Calculator:  
  
    def add(self, a, b=0, c=0):  
        print(a + b + c)  
  
calc = Calculator()  
  
calc.add(10)  
calc.add(10, 20)  
calc.add(10, 20, 30)
```

Output

```
10  
30  
60
```

Achieving Method Overloading Using *args

```
class Calculator:  
  
    def add(self, *numbers):  
        print(sum(numbers))  
  
calc = Calculator()  
  
calc.add(10, 20)  
calc.add(10, 20, 30)  
calc.add(10, 20, 30, 40)
```

Output

```
30
60
100
```

The `*args` parameter allows a method to accept any number of arguments.

Duck Typing

Python follows the principle:

"If it walks like a duck and quacks like a duck, it is treated as a duck."

Python checks whether an object has the required method instead of checking its type.

Example

```
class Dog:

    def speak(self):
        print("Dog barks")

class Cat:

    def speak(self):
        print("Cat meows")

def animal_sound(animal):
    animal.speak()

dog = Dog()
cat = Cat()

animal_sound(dog)
animal_sound(cat)
```

Output

```
Dog barks
Cat meows
```

Operator Overloading

Python allows operators to perform different operations depending on the data type.

Example

```
print(10 + 20)

print("AI " + "ML")

print([1, 2] + [3, 4])
```

Output

```
30
AI ML
[1, 2, 3, 4]
```

The `+` operator performs:

- Addition for numbers
- Concatenation for strings
- Merging for lists

Custom Operator Overloading

```
class Number:

    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value

n1 = Number(50)
n2 = Number(70)

print(n1 + n2)
```

Output

```
120
```

AI Example – Machine Learning Models

```
class MLModel:

    def train(self):
        print("Training Generic Model")

class LinearRegression(MLModel):

    def train(self):
        print("Training Linear Regression")

class DecisionTree(MLModel):

    def train(self):
        print("Training Decision Tree")

models = [LinearRegression(), DecisionTree()]

for model in models:
    model.train()
```

Output

```
Training Linear Regression
Training Decision Tree
```

AI Example – Prediction Models

```
class PredictionModel:

    def predict(self):
        print("Generic Prediction")

class SpamDetector(PredictionModel):

    def predict(self):
        print("Spam Prediction")

class FaceRecognizer(PredictionModel):

    def predict(self):
        print("Face Recognition Prediction")

models = [SpamDetector(), FaceRecognizer()]
```

```
for model in models:  
    model.predict()
```

Real-World Example – Payment Gateway

```
class Payment:  
  
    def pay(self):  
        print("Processing Payment")  
  
class CreditCard(Payment):  
  
    def pay(self):  
        print("Processing Credit Card Payment")  
  
class UPI(Payment):  
  
    def pay(self):  
        print("Processing UPI Payment")  
  
class NetBanking(Payment):  
  
    def pay(self):  
        print("Processing Net Banking Payment")  
  
payments = [CreditCard(), UPI(), NetBanking()]  
  
for payment in payments:  
    payment.pay()
```

Advantages of Polymorphism

- Improves code reusability.
 - Reduces code duplication.
 - Makes applications flexible.
 - Simplifies maintenance.
 - Supports scalability.
 - Encourages clean object-oriented design.
-

Applications of Polymorphism

Polymorphism is widely used in:

- Artificial Intelligence
 - Machine Learning
 - Deep Learning
 - Web Development
 - Payment Systems
 - Notification Systems
 - Game Development
 - Enterprise Applications
-

Summary

In today's session, we learned:

- Introduction to Polymorphism
- Method Overriding
- Method Overloading
- Why Python does not support traditional Method Overloading
- Method Overloading using default arguments
- Method Overloading using `*args`
- Duck Typing
- Operator Overloading
- AI and real-world implementation examples

Polymorphism allows a single interface to represent different implementations, making Python applications more flexible, reusable, and easier to maintain.

Practical Exercises

Practical 1 – Animal Sounds

Create a parent class:

- `Animal`

Method:

- `sound()`

Create child classes:

- Dog
- Cat

- Cow

Override the `sound()` method in each class.

Practical 2 – Calculator

Create a `Calculator` class.

Implement an `add()` method using:

- Default arguments
- `*args`

Test it with different numbers of values.

Practical 3 – Notification System

Create classes:

- EmailNotification
- SMSNotification
- PushNotification

Each class should implement a `send()` method.

Write a function that accepts any notification object and calls `send()`.

Practical 4 – AI Prediction Models

Create a parent class:

`AIModel`

Method:

- `predict()`

Create child classes:

- ClassificationModel
- RegressionModel
- ClusteringModel

Override the `predict()` method in each class.

Assignment

Develop a **Payment Processing System** using Polymorphism.

Parent Class

PaymentMethod

Method:

- process_payment()

Child Classes

- CreditCardPayment
- DebitCardPayment
- UPIPayment
- NetBankingPayment

Requirements:

- Override the process_payment() method in every child class.
- Store all payment objects in a list.
- Use a loop to process each payment without checking the object type.

Interview Questions

1. What is Polymorphism?
2. What are the types of Polymorphism in Python?
3. What is Method Overriding?
4. What is Method Overloading?
5. Why doesn't Python support traditional Method Overloading?
6. How can Method Overloading be achieved in Python?
7. What is Duck Typing?
8. What is Operator Overloading?
9. What are the advantages of Polymorphism?
10. How is Polymorphism used in AI and Machine Learning frameworks?

Next Class Preview

In the next session, we will learn the **fourth and final pillar of Object-Oriented Programming: Abstraction**.

Topics include:

- Introduction to Abstraction

- Abstract Classes
- Abstract Methods
- The `abc` Module
- Implementing Abstract Classes
- Real-world Examples
- AI & Machine Learning Examples
- Practical Exercises and Assignment